Báo cáo BTVN 7/4/2026

Chat Server (Bài 1)

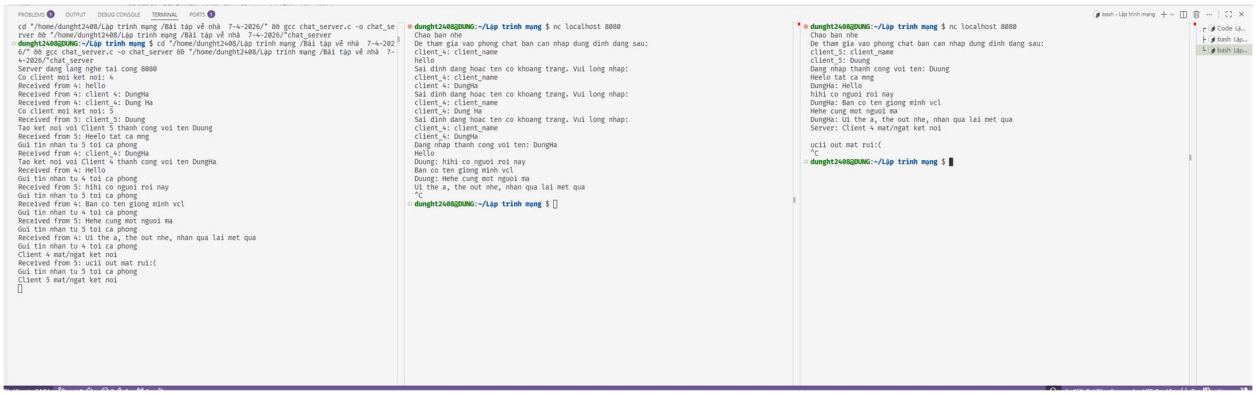
Mô tả cho Bài tập

Đây là chương trình sử dụng hàm select() để mô phỏng một ứng dụng chat giữa nhiều client với nhau. Khi một client muốn tham gia phòng, client đó cần kết nối đúng cú pháp: ***client_id: client_name***

Trong đó: *Client_id* là file descriptor của client đó.
Client_name là tên tùy chọn của người dùng, là xâu kí tự không chứa dấu cách.

Khi kết nối với server thành công, client có thể gửi tin nhắn nhập từ bàn phím tới các client khác cũng đã tham gia phòng, tin nhắn được gửi tới mọi người trong phòng sẽ được hiển thị lên với cú pháp: ***[client_name]: [Thông điệp]***

Hình ảnh 1.1:



Chương trình có kịch bản kiểm thử như sau:

Server (bên trái ảnh) bắt đầu chương trình với địa chỉ localhost (trong máy) và cổng kết nối là 8080.

Tiếp đến, có một client (ở giữa) kết nối tới server và được lắng nghe thành công. Server gửi tin nhắn yêu cầu client nhập đúng cú pháp quy định để có thể join phòng. Tuy nhiên sau lần nhập, client vẫn không gửi tin nhắn đúng định dạng, do đó vẫn chưa thể join phòng được và được Server yêu cầu nhập lại cho tới khi thành công.

Trong quá trình client_4 cố gắng join phòng, thì có một client_5 (bên phải) kết nối tới Server, thành công nhập đúng định dạng và join được phòng. Như ta có thể

thấy, client_5 đã gửi tin nhắn chào mọi người và được Server tiếp nhận và chuyển tin nhắn tới cả phòng. Nhưng vì client_4 chưa join phòng, nên tin nhắn không xuất hiện ở khung chat của client_4.

Sau đó, cuối cùng client_4 đã join phòng thành công. Và như ta có thể thấy, hai client đã thành công giao tiếp với nhau và server đã chuyển tiếp tin nhắn theo đúng định dạng quy định.

Khi một trong hai người ngắt kết nối, Server thông báo sự kiện đó tới tất cả mọi người. Như vậy ứng dụng chat đã hoạt động thành công!

Telnet Server (Bài 2)

Mô tả cho Bài tập

Đây là chương trình sử dụng hàm *select()* để mô phỏng một ứng dụng Telnet cho phép client điều khiển và thực thi lệnh hệ thống trên server từ xa. Khi một client muốn tham gia điều khiển, client đó cần kết nối và xác thực đúng cú pháp: ***[user] [password]***

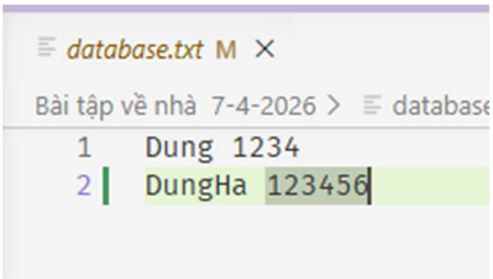
Trong đó: Server sẽ tiến hành lấy thông tin này và đối chiếu với cơ sở dữ liệu các tài khoản hợp lệ được lưu sẵn trong một file văn bản (*database.txt*).

Nếu so sánh sai hoặc sai định dạng, Server sẽ báo lỗi và không cho phép thực thi lệnh.

Khi đăng nhập thành công, client có thể gửi các lệnh hệ thống (ví dụ: ls, pwd, date). Server sẽ tiếp nhận, sử dụng hàm *system()* kết hợp kỹ thuật chuyển hướng luồng***(> out_[client_fd].txt 2>&1***) để thực thi lệnh một cách độc lập, ghi kết quả vào file văn bản tạm thời. Sau đó Server đọc file này, trả kết quả hiển thị lên màn hình cho client và tự động dọn dẹp file rác.

Hình ảnh 2.1:

Hình ảnh 2.2:



Server (bên trái ảnh 2.1) bắt đầu chương trình với địa chỉ localhost (trong máy) và cổng kết nối là 8080. File "*database.txt*" chứa danh sách tài khoản đã được chuẩn bị sẵn trong cùng thư mục.

Tiếp đến, có một client (ở giữa) kết nối tới server và được lắng nghe thành công. Server gửi lời chào và yêu cầu client nhập tài khoản mật khẩu đúng cú pháp quy định. Tuy nhiên ở lần nhập đầu tiên, client nhập sai định dạng hoặc thông tin không khớp với database, do đó chưa thể xác thực và được Server báo lỗi, yêu cầu nhập lại cho tới khi thành công.

Sau đó, client này đã nhập đúng tài khoản và đăng nhập thành công. Dấu nhắc lệnh xuất hiện, client gửi một lệnh hệ thống như "*ls*" lên Server. Như ta có thể thấy, Server đã tiếp nhận, thực thi ngầm lệnh này, lưu kết quả ra file riêng biệt cho client đó rồi gửi trả lại toàn bộ danh sách file/thư mục hiển thị trực tiếp trên màn hình của client.

Trong quá trình client đầu tiên thao tác, thì có một client thứ hai (bên phải) kết nối tới Server, thành công đăng nhập và gõ một lệnh bị sai (lệnh không tồn tại). Vì mỗi client được cấp một file kết quả riêng biệt (dựa trên client_fd), Server vẫn xử lý trơn tru, bắt được luồng thông báo lỗi của hệ điều hành và trả về đúng thông báo lỗi đó cho riêng client thứ hai mà không làm ảnh hưởng tới người còn lại.

Khi một trong hai người ngắt kết nối, Server ghi nhận sự kiện đó, dọn dẹp bộ nhớ và ngừng theo dõi. Như vậy ứng dụng Telnet Server đã hoạt động thành công.